МІНІСТЕРСТВО ОСВІТИ І НАУКИ УКРАЇНИ
ЛЬВІВСЬКИЙ НАЦІОНАЛЬНИЙ УНІВЕРСИТЕТ ІМЕНІ ІВАНА ФРАНКА

Факультет електроніки та комп’ютерних технологій

ЗВІТ
З ЛАБОРАТОРНОЇ РОБОТИ №1
Варіант 4

на тему «Основи функціонального програмування у Python»

Виконав:
студент групи ФеП-22
Корецький Андрій Вікторович

Перевірив:
аспірант
Остріков Данило Андрійович

Львів – 2026

Мета роботи. зрозуміти поняття чистої функції та референтної прозорості; навчитися виявляти та ізолювати побічні ефекти; переписати імперативний код у функціональному стилі; застосувати typing.Callable для параметризації обчислень.

Теоретичні відомості

Чиста функція — детермінована функція (однаковий вхід дає однаковий вихід), яка не змінює глобальний стан чи вхідні дані і не виконує введення-виведення.

Референтна прозорість означає, що виклик функції можна замінити його значенням без зміни поведінки програми. Це спрощує тестування, кешування та міркування про код.

Побічні ефекти — I/O (print, файли, мережа), мутація вхідних структур, використання або зміна глобальних змінних, залежність від time, random, os.environ.

Функціональний підхід полягає у відокремленні **чистого ядра** (перетворення даних) від **оболонки ефектів** (читання та виведення). Типи `Callable[[...], ...]` дозволяють передавати політики обчислень (знижки, податки, фільтри) як аргументи — це функції вищого порядку.

Індивідуальне завдання (варіант 4)

Тема «Кошик покупця». Вхідні дані — товари з полями name, price, qty, category. Потрібно обчислити subtotal, категоріальну знижку, купон і ПДВ. Правила знижки та податку передаються через Callable; результатом є новий кошик, вхідні дані не змінюються.

Хід роботи

1. Модель даних і типи політик

Позиції кошика описано через TypedDict (CartItem, PricedItem, CartSummary). Для кожної політики оголошено окремий тип Callable: фільтр позицій, категоріальна знижка, купон, ПДВ і постачальник часу.

```
class CartItem(TypedDict):
    """Позиція кошика: назва, ціна за одиницю, кількість, категорія."""

    name: str
    price: float
    qty: int
```

```
    category: str

ItemFilterFn = Callable[[CartItem], bool]

CategoryDiscountFn = Callable[[CartItem, float], float]
CouponFn = Callable[[float], float]

TaxFn = Callable[[float], float]

NowFn = Callable[[], float]
```

Лістинг 1 — фрагмент файлу core.py

2. Чисті функції предметної області

Реалізовано функції line_total, cart_subtotal, with_pricing та is_valid_item. with_pricing не змінює вхідну позицію, а повертає НОВИЙ словник із розрахованими полями line_total, discount і discounted_total.

```
def line_total(item: CartItem) -> float:
    """Вартість однієї позиції: price * qty."""
    return item["price"] * item["qty"]

def cart_subtotal(items: Iterable[CartItem]) -> float:
    """Subtotal кошика — сума вартостей усіх позицій."""
    return sum(line_total(it) for it in items)

def with_pricing(item: CartItem, discount: float) -> PricedItem:
    """Повертає НОВИЙ словник з розрахунковими полями (без мутації ``item``)."""
    total = line_total(item)
    return {
        "name": item["name"],
        "price": item["price"],
        "qty": item["qty"],
        "category": item["category"],
        "line_total": round_money(total),
        "discount": round_money(discount),
        "discounted_total": round_money(total - discount),
    }
```

Лістинг 2 — фрагмент файлу core.py

3. Фабрики політик (функції, що повертають функції)

Політики знижки, купона та ПДВ створюються фабриками. make_category_discount робить копію словника ставок, тому зміна оригіналу ззовні не впливає на вже створену політику.

```
def make_category_discount(rates: Mapping[str, float]) -> CategoryDiscountFn:
    """Категоріальна знижка: ``rates`` — частка знижки для кожної категорії.

    Копія словника робиться всередині, тож зміна оригіналу ззовні не впливає
    на вже створену політику (референтна прозорість).
    """
    frozen_rates = dict(rates)

    def discount(item: CartItem, amount: float) -> float:
        return amount * frozen_rates.get(item["category"], 0.0)

    return discount
def make_percent_coupon(percent: float, min_amount: float = 0.0) -> CouponFn:
    """Купон на відсоток від суми, діє якщо сума >= ``min_amount``."""

    def coupon(amount: float) -> float:
        return amount * (1 - percent) if amount >= min_amount else amount
```

```
    return coupon

def make_vat(rate: float) -> TaxFn:
    """ПДВ як функція: повертає суму податку для заданої бази."""
    return lambda amount: amount * rate
```

Лістинг 3 — фрагмент файлу core.py

4. Функція вищого порядку make_cart_processor

Обробник кошика збирається з переданих Callable-політик. Порядок розрахунку: subtotal → категоріальна знижка → купон → ПДВ → total. Глобальних констант у розрахунку немає — усі параметри передаються явно.

```
def make_cart_processor(
    *,
    accept: ItemFilterFn,
    category_discount: CategoryDiscountFn,
    apply_coupon: CouponFn,
    vat: TaxFn,
) -> CartProcessor:
    """Функція вищого порядку: повертає налаштований обробник кошика.

    Порядок розрахунку:
    subtotal -> категоріальна знижка -> купон -> ПДВ -> total.
    """
    def process(items: Iterable[CartItem]) -> CartSummary:
        accepted = [it for it in items if accept(it)]
        priced = [
            with_pricing(it, category_discount(it, line_total(it))) for it in accepted
        ]
        subtotal = sum(p["line_total"] for p in priced)
        cat_discount = sum(p["discount"] for p in priced)
        after_discount = subtotal - cat_discount
        after_coupon = apply_coupon(after_discount)
        vat_amount = vat(after_coupon)
        return {
            "items": priced,
            "count": len(priced),
            "subtotal": round_money(subtotal),
            "category_discount": round_money(cat_discount),
            "after_discount": round_money(after_discount),
            "coupon_discount": round_money(after_discount - after_coupon),
            "after_coupon": round_money(after_coupon),
            "vat": round_money(vat_amount),
            "total": round_money(after_coupon + vat_amount),
        }

    return process
```

Лістинг 4 — фрагмент файлу core.py

5. Оболонка побічних ефектів (app.py)

Виведення результатів винесено у функцію render_report, яка лише споживає чистий результат. Залежність від часу передається у stamp_total через Callable[[], float] (інжекція залежності), тому ядро лишається чистим.

```
def run() -> None:
    # 1) Стандартна політика
    default_result = create_default_processor()(SAMPLE_CART)
    render_report(
        "Кошик — стандартна політика (купон 5% від 1000, ПДВ 20%)", default_result
    )
```

```
    # 2) Інша політика без зміни ядра: фіксований купон, ПДВ 7%, фільтр ціни
    custom = make_cart_processor(
        accept=make_min_price_filter(400.0),
        category_discount=make_category_discount({"electronics": 0.15}),
        apply_coupon=make_fixed_coupon(500.0, min_amount=2000.0),
        vat=make_vat(0.07),
    )
    render_report(
        "Кошик — альтернативна політика (купон -500, ПДВ 7%)", custom(SAMPLE_CART)
    )
    # 3) Композиція маленьких функцій: знижка 10% + купон -100
    discount_then_coupon = compose(
        make_fixed_coupon(100.0), make_percent_discount(0.10)
    )
    print("compose(купон -100, знижка 10%)(1000) =", discount_then_coupon(1000.0))
    # 4) Інжекція часу: ядро лишається чистим, time.time — лише тут
    total, ts = stamp_total(default_result["total"], time.time)
    stamp = time.strftime("%H:%M:%S", time.localtime(ts))
    print(f"Сума {total:.2f} зафіксована о {stamp}")
```

Лістинг 5 — фрагмент файлу app.py

6. Тестування

Написано 15 pytest-тестів: референтна прозорість (двічі той самий вхід — однаковий результат), відсутність мутацій (порівняння з deepcopy), правильність розрахунку, Callable-політики, граничні випадки (порожній кошик, купон нижче порогу), compose і make_multiplier, інжекція часу.

```
def test_referential_transparency(sample_cart: list[CartItem]) -> None:
    r1 = pure(sample_cart)
    r2 = pure(sample_cart)
    assert r1 == r2  # той самий вхід -> той самий вихід

def test_no_mutation(sample_cart: list[CartItem]) -> None:
    original = deepcopy(sample_cart)
    pure(sample_cart)
    create_default_processor()(sample_cart)
    double_quantities(sample_cart)
    assert sample_cart == original  # вхід не змінюється
def test_full_calculation(sample_cart: list[CartItem]) -> None:
    r = pure(sample_cart)
    assert r["count"] == 3  # «Брак» з ціною 0 відфільтровано
    assert r["subtotal"] == pytest.approx(1600.0)
    assert r["category_discount"] == pytest.approx(120.0)  # 100 + 20 + 0
    assert r["after_discount"] == pytest.approx(1480.0)
    assert r["coupon_discount"] == pytest.approx(74.0)  # 5% від 1480
    assert r["after_coupon"] == pytest.approx(1406.0)
    assert r["vat"] == pytest.approx(281.2)
    assert r["total"] == pytest.approx(1687.2)
```

Лістинг 6 — фрагмент файлу tests/test_core.py

Результати виконання

Результат запуску програми командою `python main.py`:

```
> python main.py
================================================================
Кошик — стандартна політика (купон 5% від 1000, ПДВ 20%)
================================================================
Товар           Кат.              Ціна  К-ть      Сума   Знижка
Ноутбук         electronics   25000.00     1  25000.00  2500.00
```

```
Мишка           electronics     450.00     2    900.00    90.00
Книга Python    books           520.00     1    520.00    26.00
Кава            food            310.00     3    930.00     0.00
---------------------------------------------------------------
Позицій у розрахунку:   4
Subtotal:               27350.00
Категоріальна знижка:  -2616.00
Після знижки:           24734.00
Купон:                 -1236.70
Після купона:           23497.30
ПДВ:                   +4699.46
ДО СПЛАТИ:              28196.76
===============================================================
Кошик — альтернативна політика (купон -500, ПДВ 7%)
===============================================================
Товар           Кат.              Ціна  К-ть      Сума   Знижка
Ноутбук         electronics   25000.00     1  25000.00  3750.00
Мишка           electronics     450.00     2    900.00   135.00
Книга Python    books           520.00     1    520.00     0.00
---------------------------------------------------------------
Позицій у розрахунку:   3
Subtotal:               26420.00
Категоріальна знижка:  -3885.00
Після знижки:           22535.00
Купон:                 -500.00
Після купона:           22035.00
ПДВ:                   +1542.45
ДО СПЛАТИ:              23577.45
compose(купон -100, знижка 10%)(1000) = 800.0
Сума 28196.76 зафіксована о 11:47:07
```

Рисунок 1 — результат роботи програми (Lab_01)

Результат автоматизованого тестування `pytest -v` та статичних перевірок:

```
> black --check .
All done!
> ruff check .
All checks passed!
> mypy .
Success: no issues found
> pytest -v
============================= test session starts =============================
platform win32 -- Python 3.14.3, pytest-9.2.0, pluggy-1.7.0 -- C:\labs\Функціональне
    програмування\.venv\Scripts\python.exe
rootdir: C:\labs\Функціональне програмування\functional-programming\Lab_01
configfile: pyproject.toml
testpaths: tests
plugins: platformdirs-4.14.5
collecting ... collected 15 items
tests/test_core.py::test_line_total_and_subtotal PASSED                  [  6%]
tests/test_core.py::test_with_pricing_returns_new_dict PASSED            [ 13%]
tests/test_core.py::test_is_valid_item PASSED                            [ 20%]
tests/test_core.py::test_referential_transparency PASSED                 [ 26%]
tests/test_core.py::test_no_mutation PASSED                              [ 33%]
tests/test_core.py::test_result_items_are_new_objects PASSED             [ 40%]
tests/test_core.py::test_full_calculation PASSED                         [ 46%]
tests/test_core.py::test_coupon_not_applied_below_minimum PASSED         [ 53%]
tests/test_core.py::test_empty_cart PASSED                               [ 60%]
tests/test_core.py::test_callable_policies PASSED                        [ 66%]
tests/test_core.py::test_policy_factories PASSED                         [ 73%]
tests/test_core.py::test_policy_does_not_depend_on_external_dict_changes PASSED [ 80%]
tests/test_core.py::test_compose_and_multiplier PASSED                   [ 86%]
tests/test_core.py::test_map_filter_helpers PASSED                       [ 93%]
tests/test_core.py::test_stamp_total_uses_injected_clock PASSED          [100%]
============================== 15 passed in 0.16s ==============================
```

Рисунок 2 — результати тестів і перевірок якості коду (Lab_01)

Висновок

У ході роботи реалізовано обробку кошика покупця у функціональному стилі. Розрахункове ядро складається з чистих функцій без глобального стану, мутацій і введення-виведення; уся взаємодія з консоллю зосереджена в app.py. Політики фільтрації, категоріальної знижки, купона та ПДВ передаються через typing.Callable, а функція вищого порядку make_cart_processor повертає налаштований обробник, тому правила легко замінити без зміни ядра. Тести підтверджують референтну прозорість, відсутність мутацій і правильність обчислень; код проходить mypy (strict), black та ruff.